ARM 305 – Cloud, Dev, Edge and Fog Computing | Mini Project

Edge/Fog/Cloud Task Placement Simulator

Rule-based placement, simulated VMs, tenant security and QoS results

Team member name- Yash Kumar , Yug Katyal

Where should each task run?

Edge

4 CPU, 8 GB, 5 ms
Closest to users, smallest

Fog

16 CPU, 32 GB, 25 ms
Mid-sized

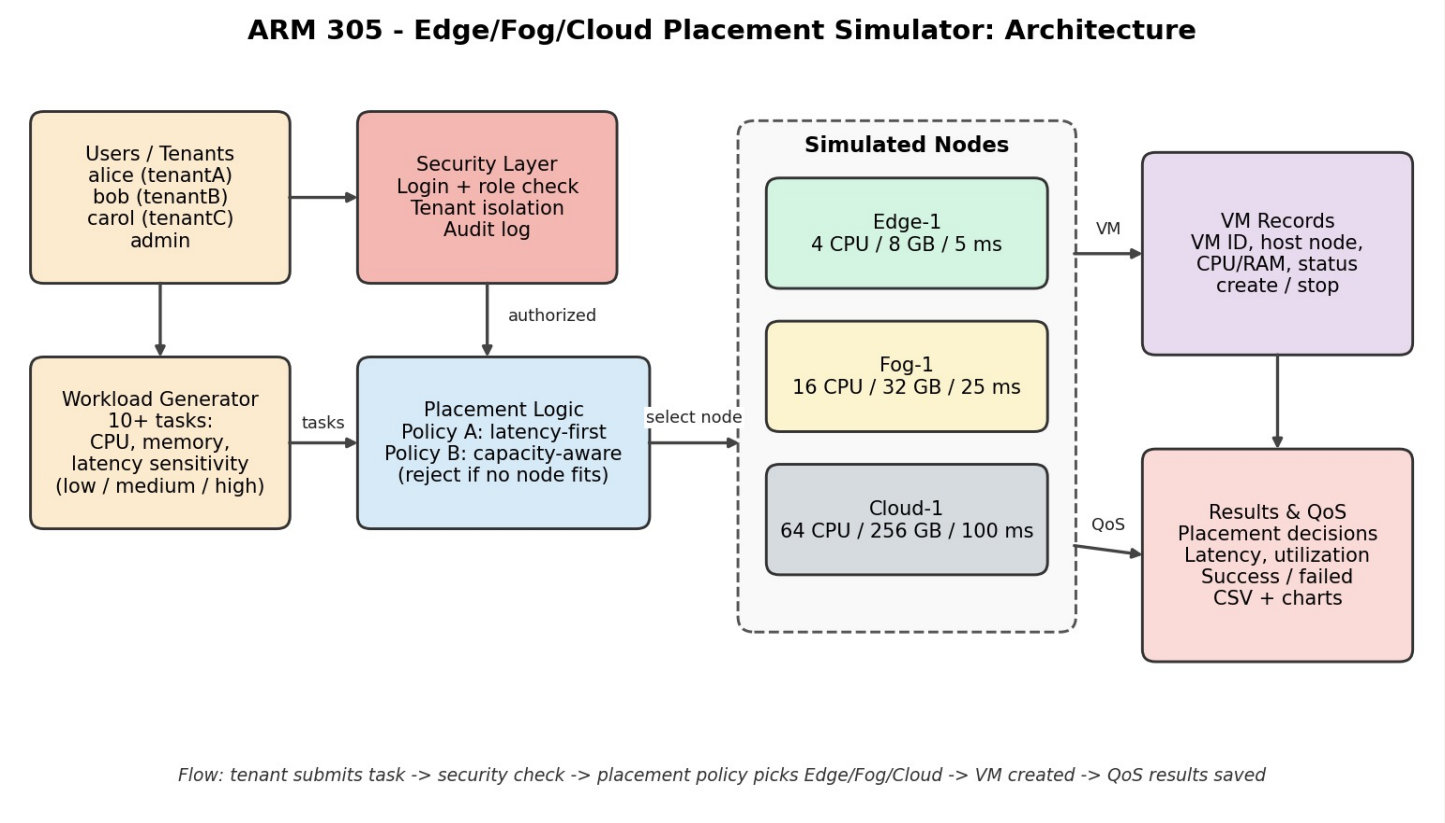
Cloud

64 CPU, 256 GB, 100 ms
Largest, farthest

Goal: a local Python simulator that places tasks, tracks VMs, isolates tenants and reports QoS.

SLA limits: high 20 ms, medium 50 ms, low 200 ms. Latency = base × (1 + utilization).

Architecture



Two placement policies

A: Latency-first

1. Sort nodes by latency
2. Take the first node with free CPU and RAM
3. Otherwise reject

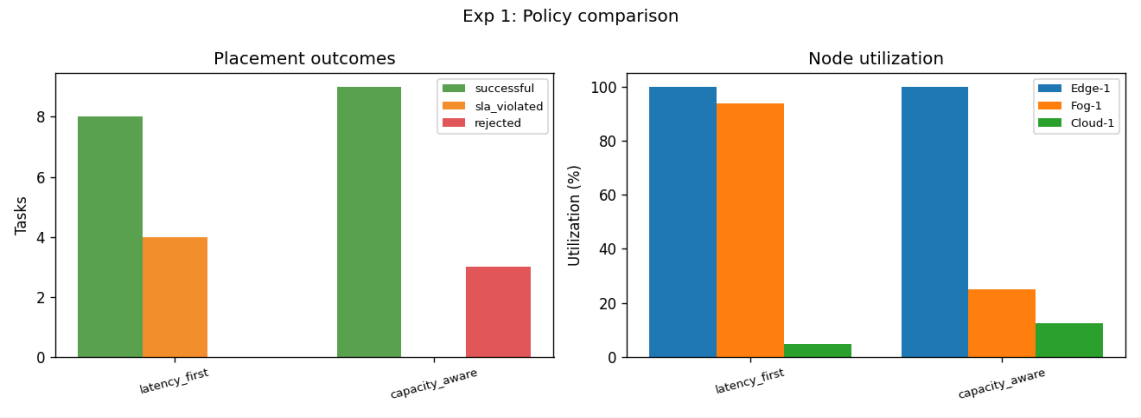
Ignores the task's SLA.

B: Capacity-aware

1. Keep nodes that fit CPU and RAM
2. Keep nodes meeting the SLA limit
3. Pick the lowest utilization
4. Otherwise reject

Experiments 1 and 2: policy comparison

Scenario	Policy	Success	SLA violated	Rejected	Success rate
12 tasks	A latency-first	8	4	0	66.7%
12 tasks	B capacity-aware	9	0	3	75.0%
25 heavy (overload)	A latency-first	9	11	5	36.0%
25 heavy (overload)	B capacity-aware	13	0	12	52.0%



B never breaks an SLA but rejects tasks no node can serve in time. A overloads Edge and Fog, then violates SLAs.

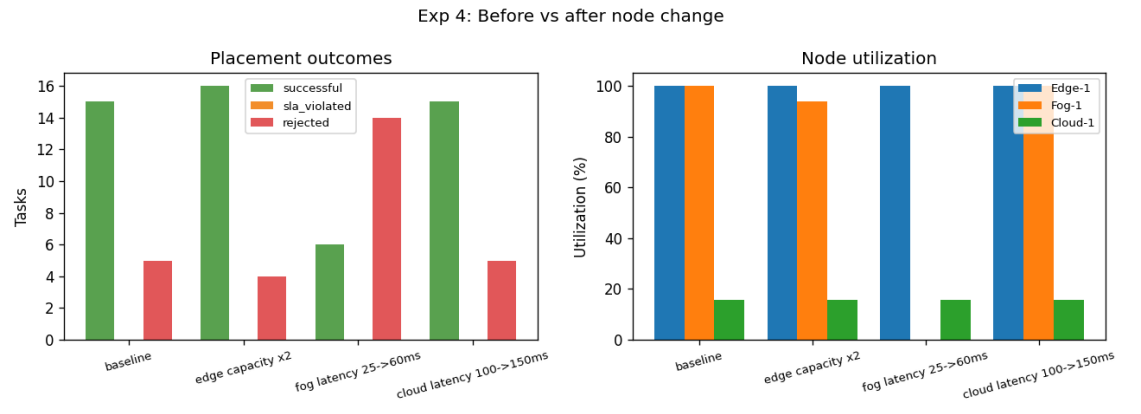
Experiments 3 and 4: security and node changes

Unauthorized access

alice viewing tenantB's T02: DENIED
alice stopping tenantB's VM002: DENIED
Own tenant and admin: ALLOWED

Before / after (success rate)

Baseline 75% | Edge capacity ×2: 80%
Fog latency 25→60 ms: 30%
Cloud latency 100→150 ms: 75%



Limitations and conclusion

- Simple latency formula; no real network or queueing
- Tasks have no duration; one node per tier
- Greedy rules; no retry or global optimization
- Demo security only; no real authentication

Capacity-aware placement trades more rejections for zero SLA violations.